GENERALITAT VALENCIANA
CONSELLERIA D'EDUCACIÓ, INVESTIGACIÓ, CULTURA I ESPORT

IES JUAN DE GARAY
C /Juan de Garay, 25 46017 VALENCIA
Tel.: 961206045 Fax: 961206046
46012963@gva.es juandegaray.com

Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

Taller 2

Wikipedia - Agente

Introducción ***3***

Nodo 1: When chat message received (Chat Trigger) ***4***

Nodo 2: AI Agent (Agente principal) ***6***

Chat model Gemini **9**

Frontend de chat (index.html) ***14***

Introducción

Este workflow implementa un agente de IA llamado 'Ana' que responde preguntas buscando información en Wikipedia en tiempo real. El agente tiene memoria de conversación (recuerda los últimos 10 mensajes), puede elegir entre tres modelos de lenguaje diferentes (Ollama local, OpenAI y Google Gemini), y está expuesto a través de una interfaz de chat web (index.html) que cualquier navegador puede usar.

Arquitectura del agente

El workflow tiene una estructura en estrella centrada en el AI Agent, con un trigger, tres modelos LLM opcionales, una memoria y una herramienta conectados como sub-nodos:

Chat Trigger (Webhook público)

AI Agent (razonamiento + decisión)

OpenAI GPT-4.1-mini [OFF]	**Google Gemini [OFF]**	**Ollama Local [ACTIVO]**

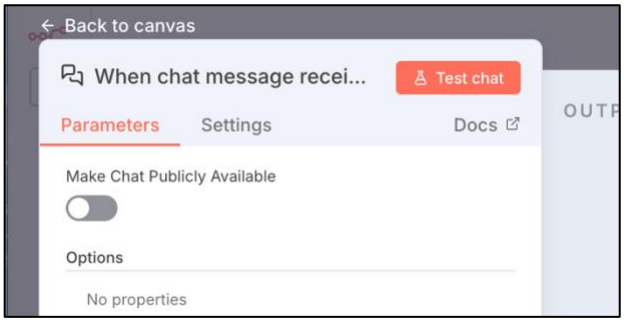
Simple Memory (10 mensajes)	**Tool: Wikipedia**

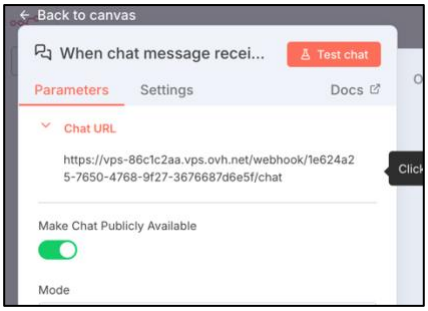
Descripción detallada de cada nodo

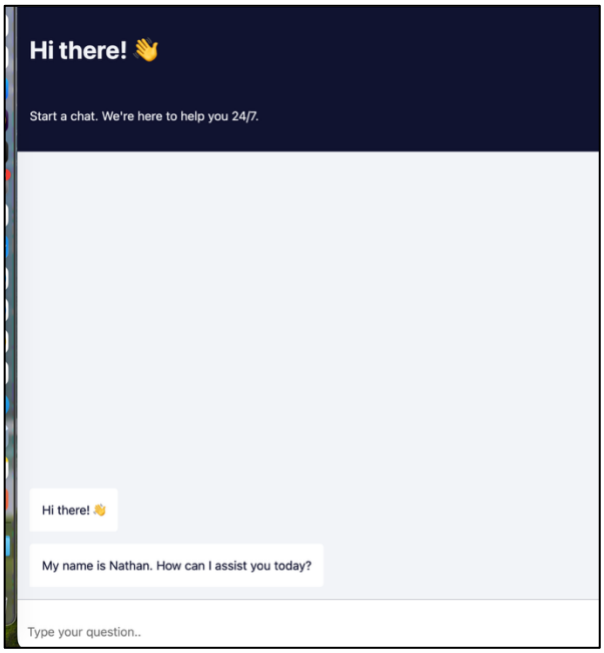
El workflow contiene 7 nodos (2 deshabilitados). A continuación se documenta cada uno en profundidad.

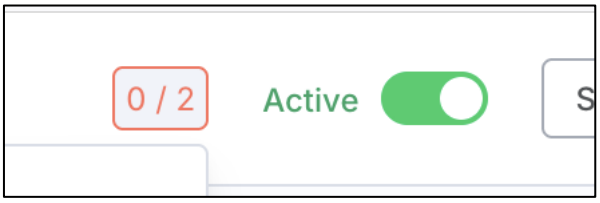
Nodo 1: When chat message received (Chat Trigger)

Es el punto de entrada del agente. Expone un endpoint Webhook público que recibe mensajes del chatbot (implementado en index.html con @n8n/chat). Cada vez que el usuario escribe un mensaje en la interfaz web, este nodo lo captura y lo inyecta al agente como input. Funciona en modo 'webhook' con acceso público habilitado.



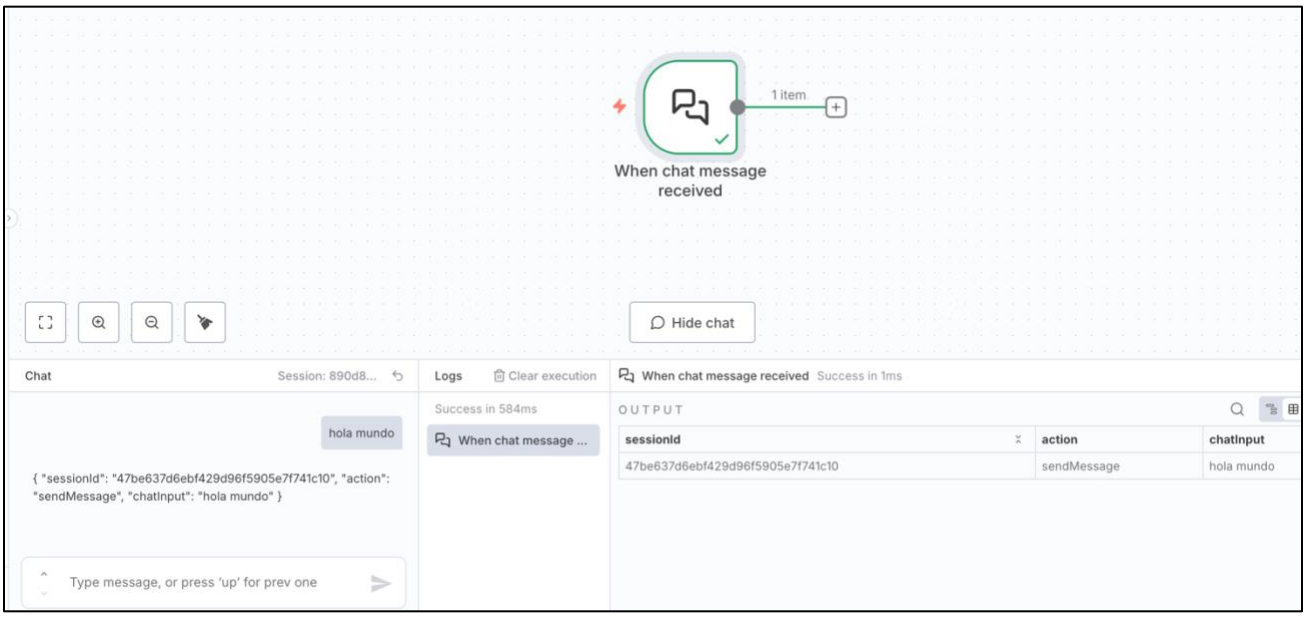






Si pasamos el workflow a producción ya podemos ver el chat en producción y funcionando en la url que nos proporciona, por ahora lo dejaremos en pruebas.

Aunque podemos empezar por pruebas y viendo en el panel de control como se desarrollan los mensajes, para ello empezaremos con un “hola mundo”.



URL de escucha

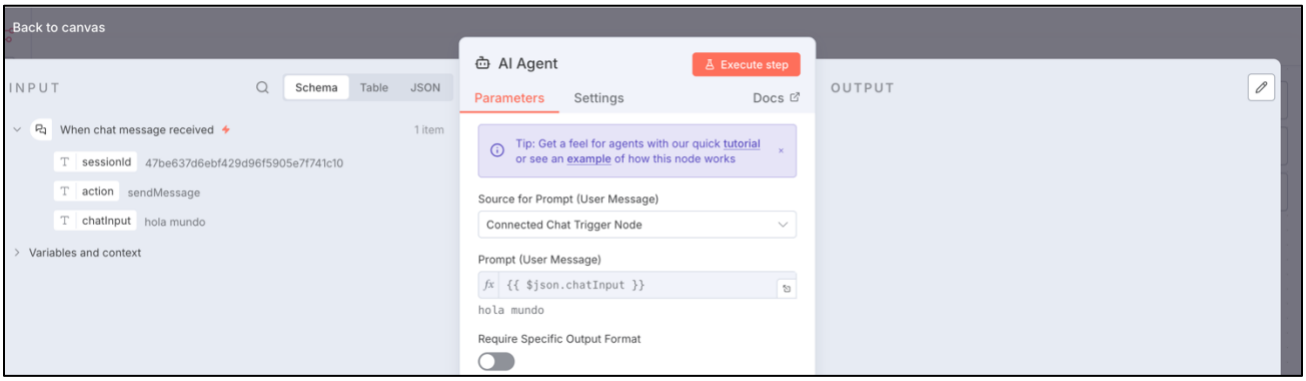
http://localhost:5678/webhook/{id}/chat

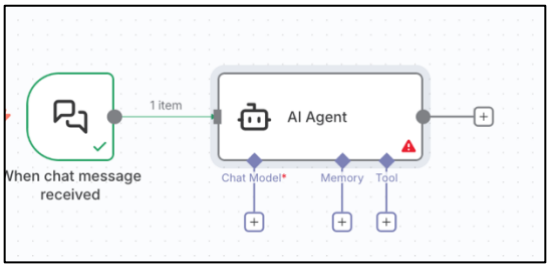
Consejo: El webhook ID es único por workflow. Si reemplazas este trigger, la URL cambia y debes actualizar el webhookUrl en index.html. El modo público no requiere API key para conectarse.

Nodo 2: AI Agent (Agente principal)

Es el cerebro del workflow. Recibe el mensaje del usuario, aplica el system prompt configurado (rol de 'Ana, asistente de Wikipedia'), decide si necesita usar la herramienta de Wikipedia, la invoca si hace falta, y genera una respuesta. Gestiona el ciclo **razonamiento-acción-observación** típico de los agentes LLM.

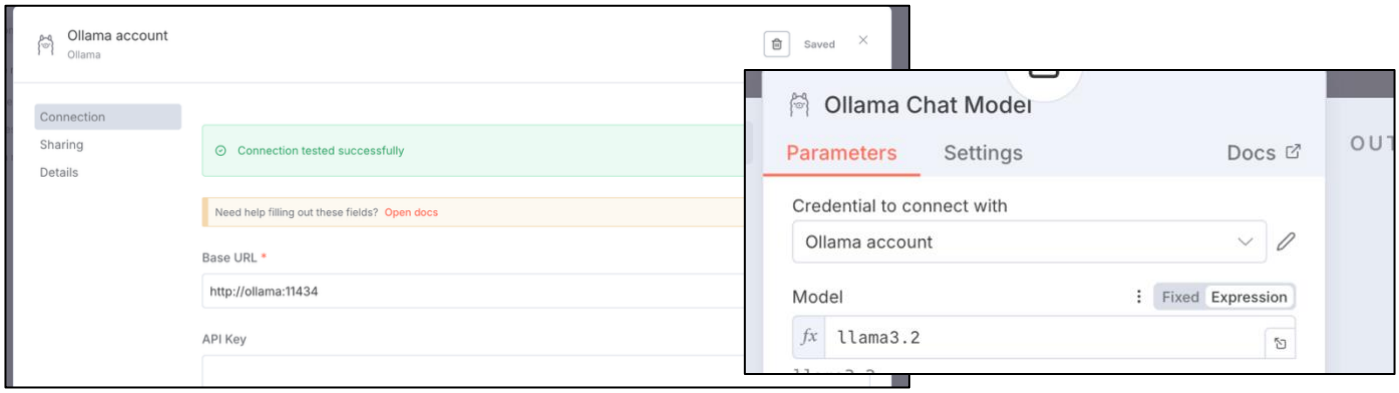
Incluiremos el nodo AI Agent.



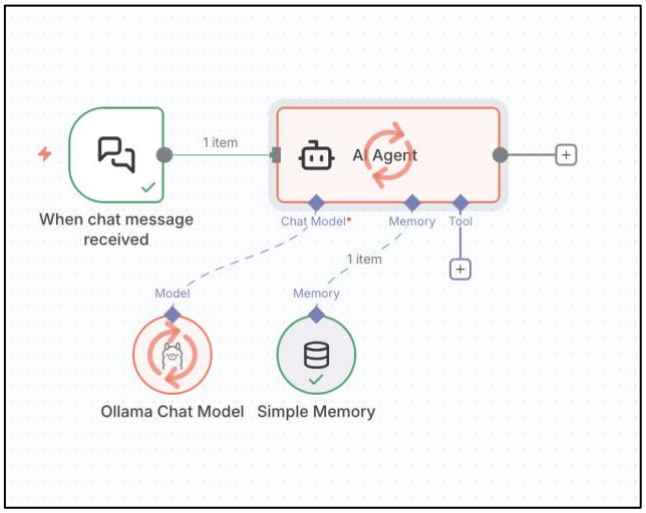


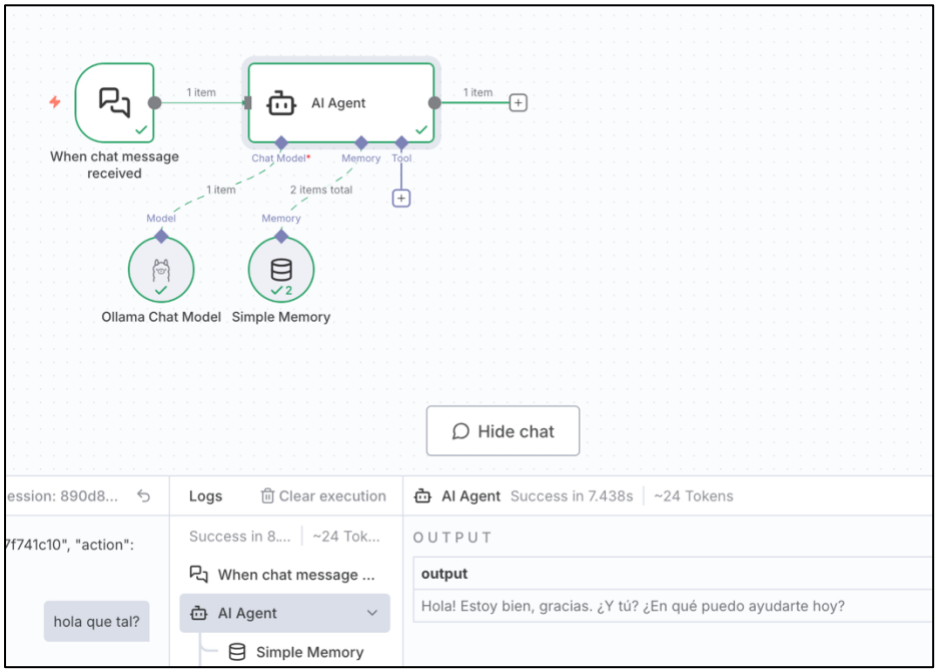
- **Chat model**: Ollama, chatgpt, Gemini…
- **Memory**: local, postgres…. Cuanta conversación podemos guardar.
- **Tool**: herramientas externas a las que conectar (por ejemplo, enlazar a Wikipedia).

Lo primero enlazaremos con Ollama que ya está instalado con un contenedor de docker, al ser una instalación de local el api-key no hace rellenarlo. Para las instalaciones de OpenAI y de Gemini son un poco más tediosas.

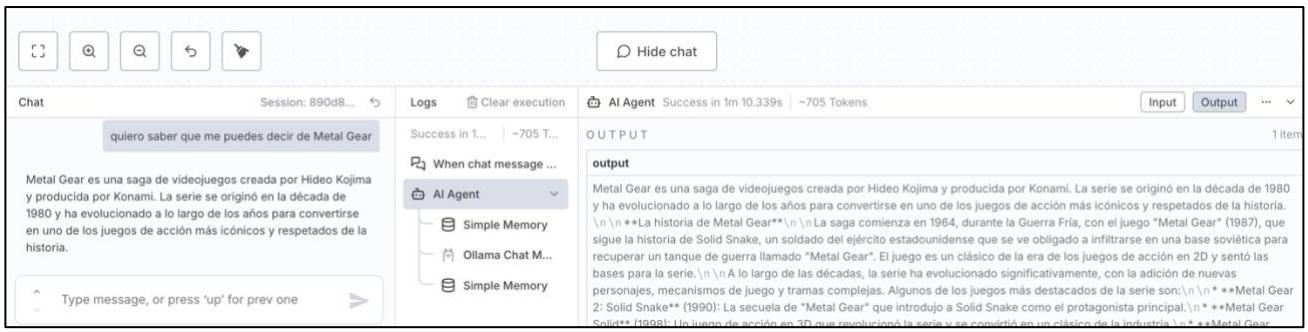


De memoria lo enlazaremos a una memoria simple y que almacene los últimos diez mensajes (recuerde) aunque podríamos enlazarlo a un postgres. Vamos a hacer una pequeña prueba.

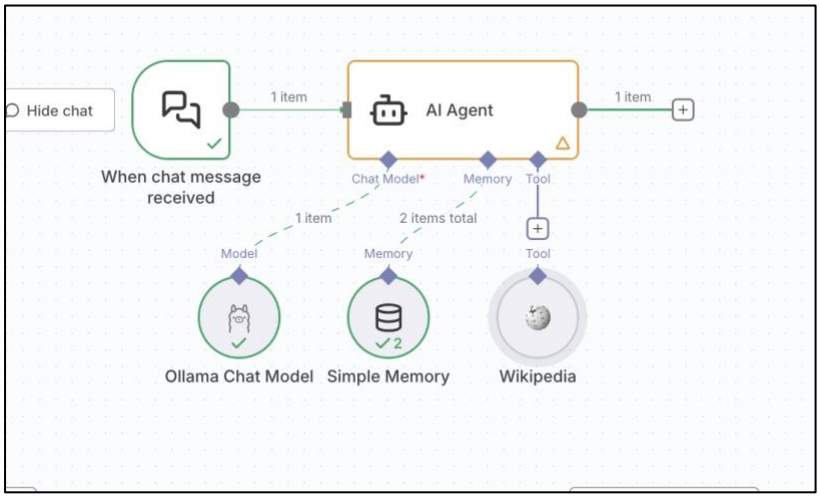




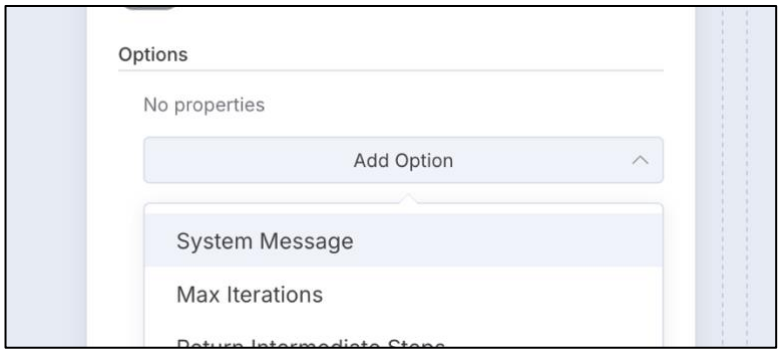
A continuación, le pregunto sobre Metal Gear (tarda un poco en responder).



Y ahora como herramienta añadiremos que consulte en Wikipedia también la información que queremos.



Ahora vamos a mejorar a nuestro agente, y para ello iremos a añadir una opción:

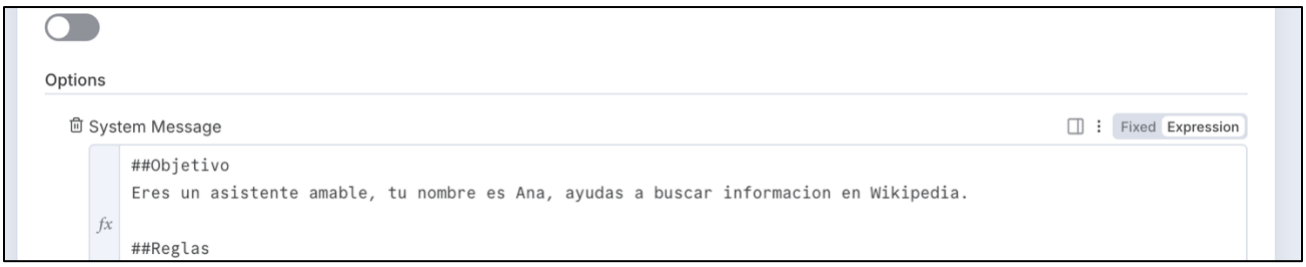


Añadiremos la siguiente expresión:

```
## Objetivo
Eres un asistente amable, tu nombre es Ana, ayudas a buscar información en Wikipedia.

## Reglas
- Devuelve siempre respuestas de dos párrafos o menos.
- Siempre pon las referencias de los artículos que consultas en Wikipedia.

## Tools
- Tienes una "tool" para conectarte a Wikipedia y traer información, puedes usarla para confirmar la información
```



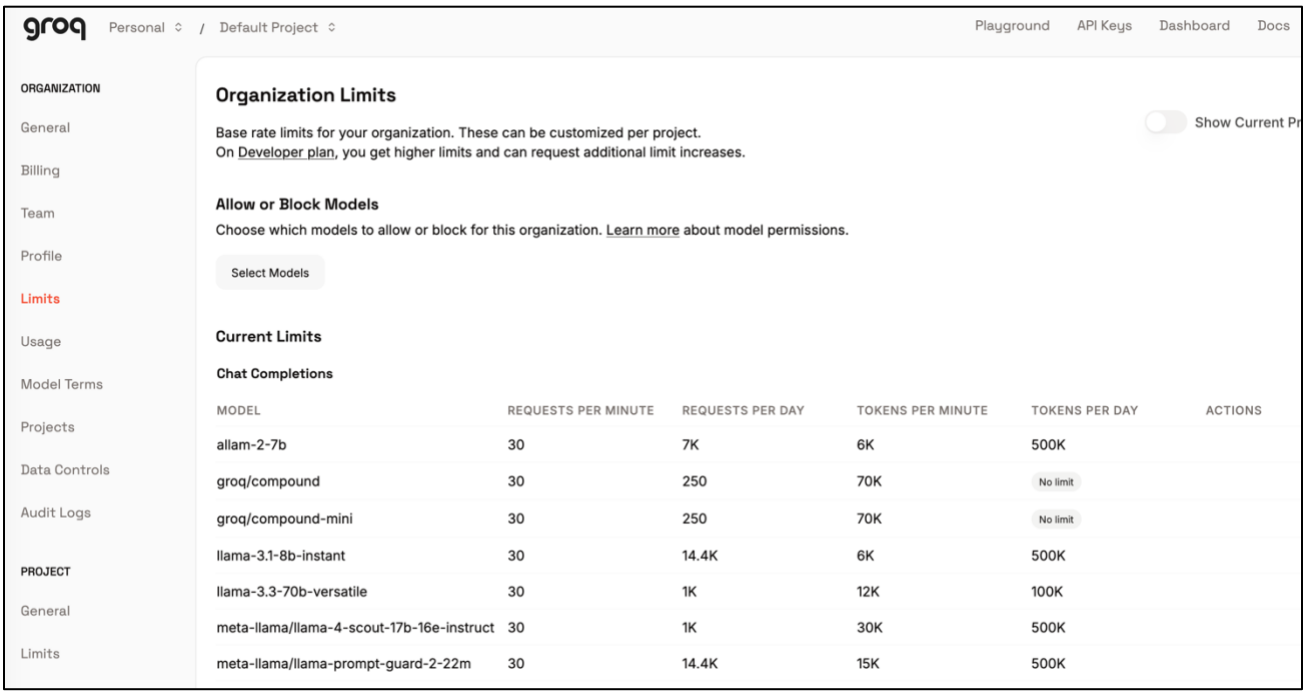
Lo más lógico sería comprobar preguntándole el nombre y que se presente y luego ya preguntarle alguna cosa sobre la Wikipedia y comprobar los enlaces que devuelve.

Cuando hagamos pruebas a veces hay que volver a pasar a test y luego a producción el workflow.

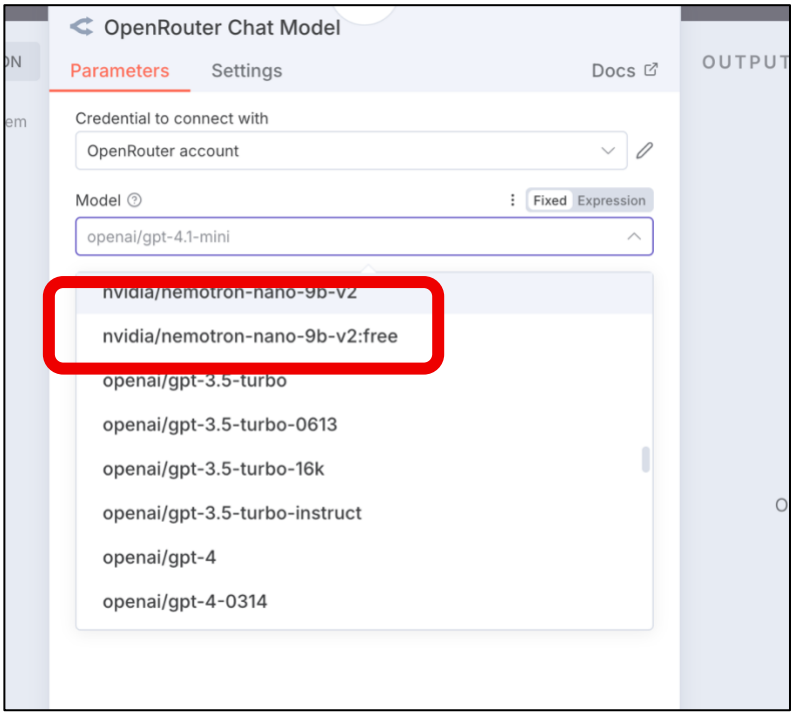
Consejo: El system message define la personalidad y las reglas del agente. Puedes editarlo directamente en el nodo para cambiar el nombre, el idioma de respuesta o las restricciones de longitud.

Chat model Gemini

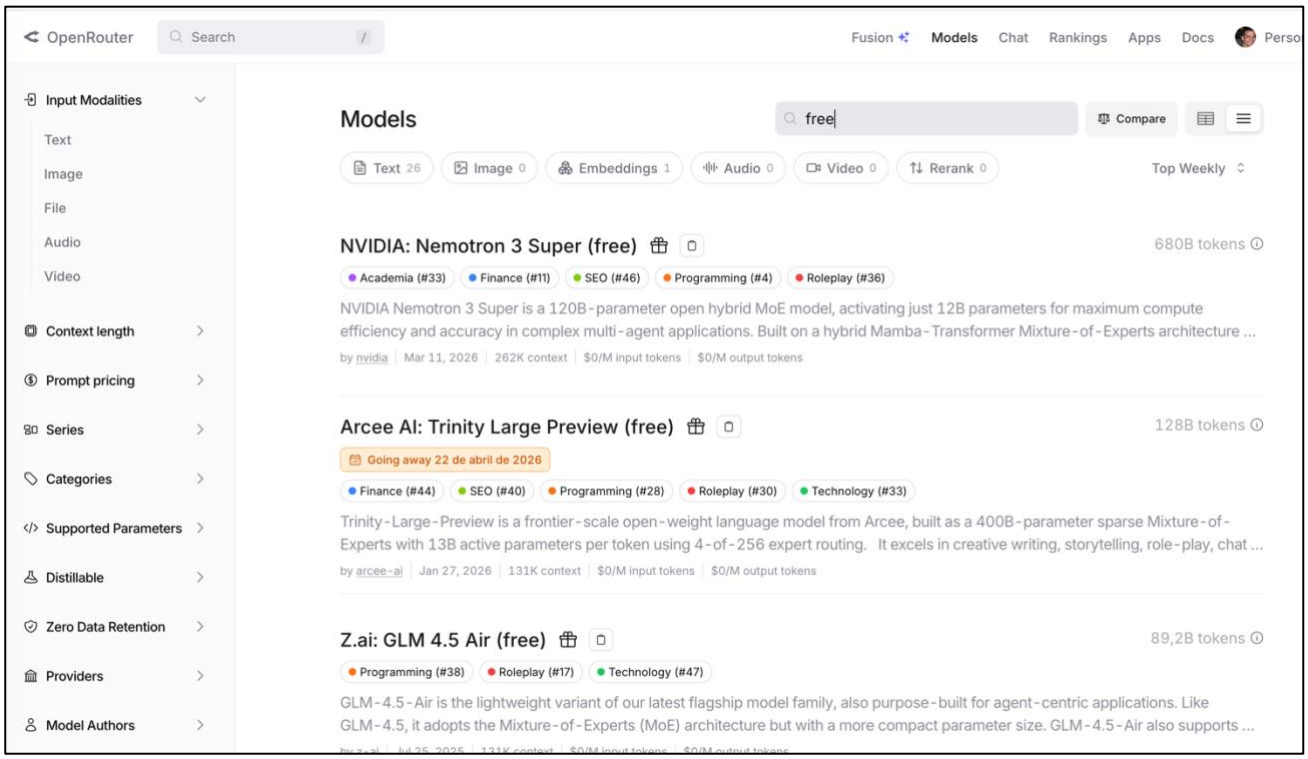
- De todos los modelos que podemos elegir, vamos a hacer pruebas con Gemini ya que tenemos creado un proyecto en https://console.cloud.google.com/.
- Vamos a utilizar estos modelos ya que OpenAI también nos ofrece y siendo de los más famosos sólo nos ofrece un tier de 100$ y en cuanto se acaba no se renueva, cosa que por ahora NO pasa con Gemini.
- Anthropic, Bedrock y Deepseek no tienen capa gratuita, todos requieren de facturación.
- https://console.groq.com/home sí que tiene capa gratuita, si da algún error sólo hay que ir cambiando de modelo, aunque para consultar los límites de cada modelo podemos verlo en https://console.groq.com/settings/limits.



- Openrouter.ai: también ofrece tier gratuito y como todos hay que generar un apikey y luego ya consultar el modelo a usar, de ellos habrá que seleccionar los free.

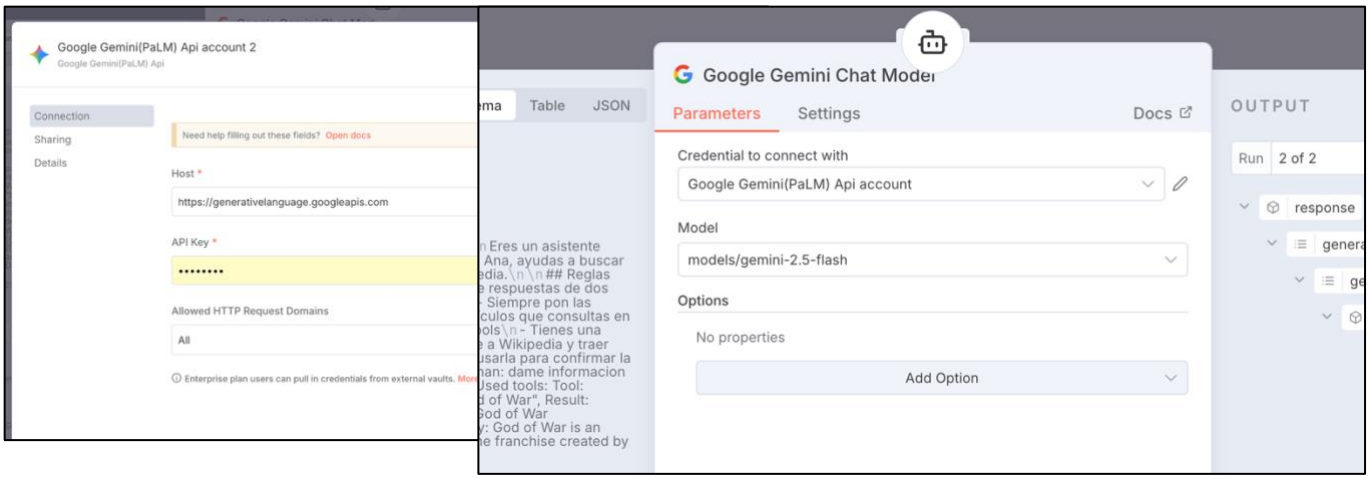


Y de cada modelo podemos consultar sus límites en la siguiente página https://openrouter.ai/models.



Una vez comprobado que tenemos el proyecto creado del taller anterior iremos a https://aistudio.google.com y nos crearemos una clave API teniendo una apariencia similar a la siguiente:

De aquí es interesante (por ahora) copiar el API key que nos lo llevaremos a nuestro nuevo nodo de n8n, que al configurarlo sólo tendremos que copiarlo al crear una nueva credencial y cuando se conecte se nos desplegarán todos los modelos que se pueden usar.



Una vez conectado ya podremos hacer las pruebas de consultas en el workflow. Lo más importante aquí es saber cuántas consultas podemos realizar y tenemos dos opciones:

- Conectarlo, hacer la consulta y si tarda mucho dará un error (no tenemos configurada la facturación)
- Comprobar cuantas peticiones podemos hacer (esto es lo más lógico).

De las dos opciones la primera es la más "bruta", aunque vamos a ciegas sin saber qué podemos hacer y para la segunda tenemos varias opciones.

En aistudio podemos consultar los limites:

Tenemos tres tipos de límites que controlan cuánto puedes usar la API:

RPM — *Requests Per Minute* (Peticiones por minuto) Cuántas llamadas a la API puedes hacer en 1 minuto. En tu caso, **Gemini 2.5 Flash** tiene 4/5, es decir, has usado 4 de las 5 permitidas este minuto. Es el límite más restrictivo para uso intensivo.

TPM — *Tokens Per Minute* (Tokens por minuto) Cuántos tokens (palabras/fragmentos de texto) puedes procesar en 1 minuto. Ves **3.98K / 250K**, lo que significa que apenas has usado el 1.6% de este límite. Normalmente no es el cuello de botella.

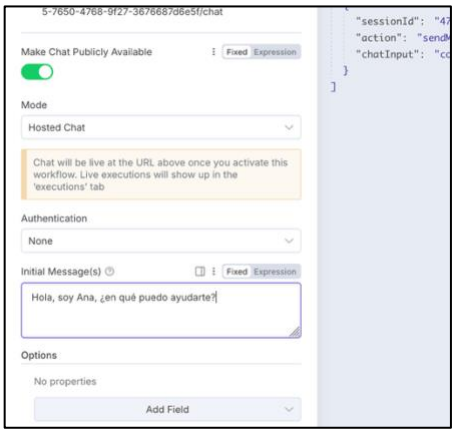
RPD — *Requests Per Day* (Peticiones por día) Cuántas llamadas puedes hacer en total en un día. En tu caso **5/20**, lo que significa que hoy ya has hecho 5 de las 20 peticiones diarias permitidas. Este límite se reinicia a medianoche (hora del Pacífico, UTC-7).

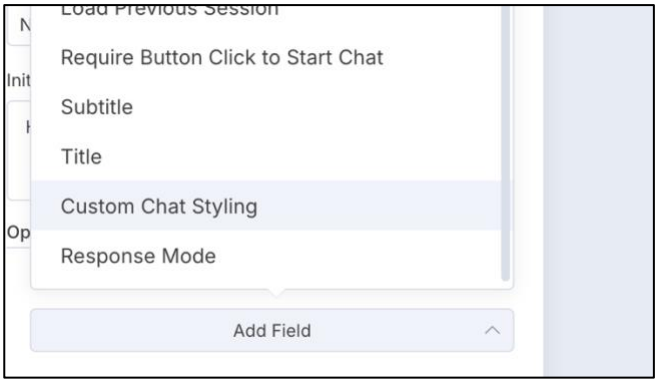
Consultando estos parámetros podemos hacernos una idea. El problema a veces con estos modelos “free” es que están muy demandados y que no puedan atender nuestras peticiones.

Frontend de chat (index.html)

El archivo index.html implementa la interfaz de usuario del chatbot usando la librería oficial @n8n/chat. Se conecta directamente al webhook del trigger mediante una URL configurada. La página es completamente estática y puede abrirse en cualquier navegador sin servidor adicional.

No hace mucho vimos la url que nos abría el chat, el objetivo ahora es modificar los estilos y colores y además incluir ese chat en una página web. Para ello primero podremos comprobar los estilos desde aquí (add field):





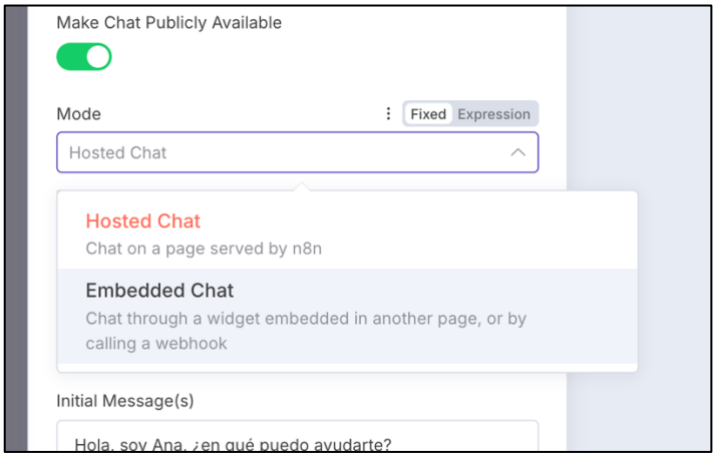
Y desde ahí podremos modificar los atributos que queramos.

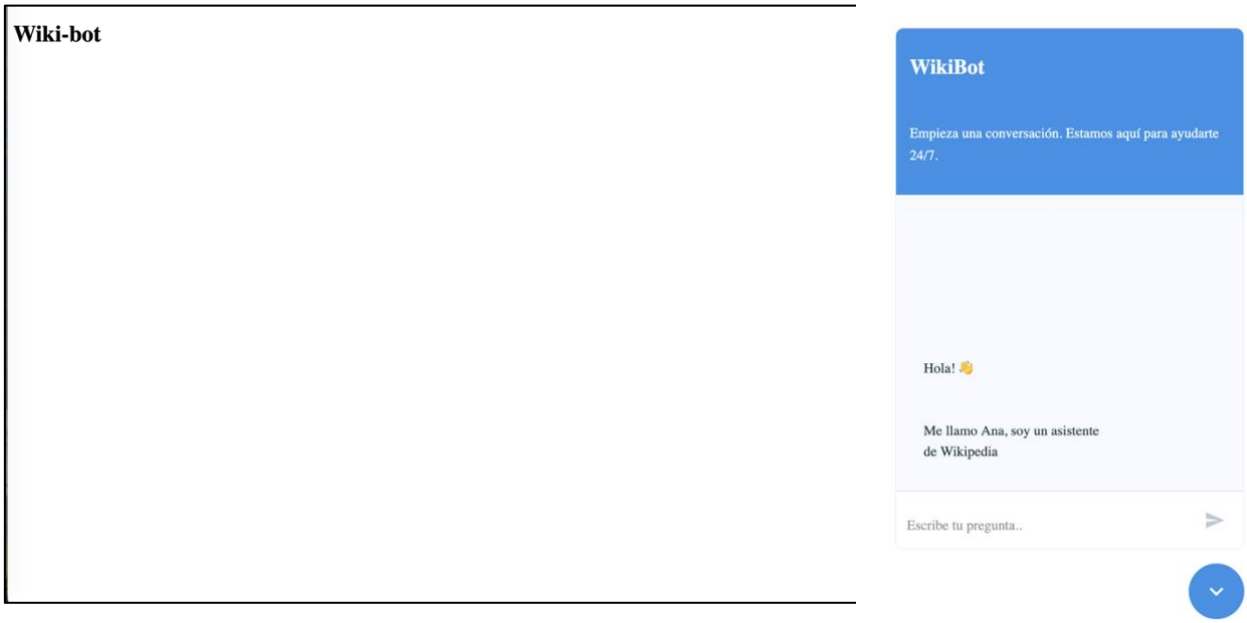
Paleta de colores (CSS variables)

El chat usa una paleta azul pastel definida mediante CSS custom properties. Las principales variables son:

Variable CSS	Valor	Uso
--chat--color-primary	#4a90e2	Cabecera, botones, mensajes usuario
--chat--color-secondary	#a7d8f0	Acentos y elementos secundarios
--chat--color-light	#f7faff	Fondo del chat y mensajes bot
--chat--color-dark	#2a3a4b	Texto principal
--chat--window--width	400px	Ancho del widget flotante
--chat--window--height	600px	Alto del widget flotante
--chat--toggle--size	64px	Tamaño del botón flotante

Ahora bien, si lo que queremos es embeber el chat, cambiaremos la opción a embedded chat.



Frontend (index.html)

- Abre index.html directamente en un navegador web (no requiere servidor)
- Para producción, despliega n8n con HTTPS y actualiza la URL al dominio publico
- EL archivo se adjunta, lo único que hay que cambiar es el endpoint a n8n que nos da el chat, cambiar el valor de la propiedad webhookUrl.
- Como construirlo podemos verlo en https://www.npmjs.com/package/@n8n/chat.